

Reviewer Pack — Minimal Rank of Ehrhart Cohomology Bounds Circuit Lower Boun...

Entry 5097e7a8ad5d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 18:39:25 UTC

Minimal Rank of Ehrhart Cohomology Bounds Circuit Lower Bound for Randomized OR

Entry ID: 5097e7a8ad5d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 18:39:25 UTC

1. Conjecture

Field A (mathematical branch): Ehrhart Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

[‘For a given boolean function f with n variables, the minimal rank of its associated Ehrhart cohomology group $H_1(f)$ is upper-bounded by the randomized OR complexity of f , i.e., for all instances with size $n \leq 40$, $\text{ROR}_f(n) \leq c * |H_1(f)|^2$, where c is a constant.’, ‘Here, $\text{ROR}_f(n)$ denotes the randomized OR complexity of the boolean function f , defined as the minimum number of random bits needed to evaluate f for all possible inputs in the worst case.’, ‘For any instance with size $n \leq 40$, if there exists an input assignment that requires more than $c * |H_1(f)|^2$ random bits to evaluate f , then the conjecture is falsified.’]

Rationale (proposer’s reasoning):

[‘Ehrhart Theory studies the relationship between integer points in polytopes and algebraic varieties. It has been applied in enumeration problems, but not extensively in complexity theory.’, ‘The Ehrhart cohomology group $H_1(f)$ captures the combinatorial information of a boolean function f and is expected to be related to circuit lower bounds.’, ‘By connecting these fields, we may uncover new insights into the nature of boolean functions and their complexity.’]

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cf1eddab64b88db1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For all boolean functions with n variables ($n \leq 40$), if the slope of the best fit line between $|H_1(f)|^2$ and $\text{ROR}_f(n)$ is less than or equal to a constant c , then the conjecture is supported. Otherwise, if there exists an instance where $\text{ROR}_f(n)$ exceeds $c * |H_1(f)|^2$ for any $n \leq 40$, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - (Ehrhart Theory AND Boolean Circuit Complexity) AND (circuit lower bound OR randomized OR complexity) - (H₁(f) AND Ehrhart cohomology) AND (Boolean circuit complexity OR ROR_f(n)) - minimal rank H₁(f) upper bound randomized OR complexity Boolean circuits

Top relevant hits considered: - [<http://arxiv.org/abs/2504.19966v3>] Quantum circuit lower bounds in the magic hierarchy - [<http://arxiv.org/abs/1903.10609v3>] Circuit Complexity of Knot States in Chern-Simons theory - [<http://arxiv.org/abs/1407.6169v3>] Multiplicative Complexity of Vector Valued Boolean Functions - [<http://arxiv.org/abs/1812.02106v2>] First Measurements of the Double-Polarization Observables F , P , and H in ω Photoproduction off Transversely Polarized Targets - [<http://arxiv.org/abs/2501.06483v2>] Study of light-meson resonances decaying to $K_S^0 K \pi$ in the $B \rightarrow (K_S^0 K \pi) K$ channels - [<http://arxiv.org/abs/1202.2652v2>] Ehrhart f^* -coefficients of polytopal complexes are non-negative integers - [<http://arxiv.org/abs/2603.25938v1>] Narrowband searches for continuous gravitational waves from known pulsars in the first two parts of the fourth LIGO-Virgo O3 observing run - [<http://arxiv.org/abs/2512.17990v2>] Constraints on gravitational waves from the 2024 Vela pulsar glitch

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def ehrhart_cohomology_group_size(f):
        n = int(math.log2(len(f)))
        # Simplified computation for demonstration purposes
        return n

    def randomized_or_complexity(f):
        n = len(f)
        max_bits = 0
```

```

    for _ in range(100): # Sample 100 random inputs
        input_bits = ''.join(str(random.choice([0, 1])) for _ in range(n))
        bits_required = sum(1 for i, bit in enumerate(input_bits) if f[i] == int(bit))
        max_bits = max(max_bits, bits_required)
    return max_bits

n_values = [40]
results = []

for n in n_values:
    f = generate_random_boolean_function(n)
    h1_size = ehrhart_cohomology_group_size(f)
    ror_f_n = randomized_or_complexity(f)
    results.append((h1_size**2, ror_f_n))

if len(results) < 30:
    return {
        "metric_name": "ROR_f(n)",
        "metric_value": None,
        "instances_tested": len(results),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

h1_squared, ror_f_n = zip(*results)
slope, intercept = linear_regression(h1_squared, ror_f_n)

return {
    "metric_name": "ROR_f(n)",
    "metric_value": slope,
    "instances_tested": len(results),
    "conjecture_holds": slope <= 0.5, # Placeholder constant c
    "counterexample": "" if slope <= 0.5 else f"Counterexample found with slope {slope}"
}

def linear_regression(x, y):
    n = len(x)
    sum_x = sum(x)
    sum_y = sum(y)
    sum_xy = sum(xi * yi for xi, yi in zip(x, y))
    sum_xx = sum(xi**2 for xi in x)

    slope = (n * sum_xy - sum_x * sum_y) / (n * sum_xx - sum_x**2)
    intercept = (sum_y - slope * sum_x) / n

    return slope, intercept

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

```

```

if all(result["conjecture_holds"] for result in results):
    mean_slope = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = 1.0
    print(f"RESULT: SUPPORTED mean={mean_slope} std=0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    counterexample = result["counterexample"]
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot determine if the conjecture is supported or falsified based on the pre-registered support condition. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12907
2	preregistration	ollama_remote	glm4:latest	0	0	5991
3	novelty	ollama_remote	glm4:latest	0	0	4921
4	novelty	ollama_remote	glm4:latest	0	0	6336
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15655
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12509
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10411
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15542
9	judge	ollama_remote	glm4:latest	0	0	11513

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 95786 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/5097e7a8ad5d.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5097e7a8ad5d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5097e7a8ad5d.tar.gz` (if generated)